

1. Types of Collections in C#

A) Non-Generic Collections (old, from `System.Collections`)

- Store object, so boxing/unboxing happens with value types.
- Less type safety.
- Used in older .NET code.

Examples:

- ArrayList
- Hashtable
- Queue
- Stack
- SortedList

B) Generic Collections (from `System.Collections.Generic`)

- Strongly typed (no boxing/unboxing).
- More efficient and type safe.
- Preferred in modern .NET.

Examples:

- List<T>
- Dictionary<TKey, TValue>
- Queue<T>
- Stack<T>
- SortedList<TKey, TValue>
- HashSet<T>
- LinkedList<T>

2. Generic vs Non-Generic

FEATURE	GENERIC (LIST<T>, DICTIONARY<TKEY, TVALUE>)	NON-GENERIC (ARRAYLIST, HASHTABLE)
TYPE SAFETY	Strongly typed	Stores object (unsafe casting)
PERFORMANCE	No boxing/unboxing	Boxing/unboxing for value types
MODERN USAGE	Recommended	Legacy, avoid
FLEXIBILITY	Works with any type	Works with objects only

3. Main Collection Types – Features, Pros & Cons

TYPE	USED FOR	FEATURES	✓ PROS	✗ CONS	RECOMMENDED CASE
ARRAY	Fixed-size storage	Fast indexing	Very fast access	Fixed size	When size is known & performance critical
ARRAYLIST (NON-GENERIC)	Dynamic array	Stores object	Auto resize	Boxing/unboxing	Legacy code, avoid in new
LIST<T>	Dynamic array (generic)	Strongly typed, resize able	Fast lookup by index, LINQ support	Slower insert in middle (shifts items)	General-purpose replacement of ArrayList
LINKEDLIST	Doubly linked nodes	Efficient insert/remove	Fast insert/delete anywhere	Slow random access (no index)	When lots of inserts/removes in middle
QUEUE / QUEUE<T>	FIFO (First In, First Out)	Enqueue/ Dequeue	Simple FIFO, Fast	No random access	Message buffers, task scheduling
STACK / STACK<T>	LIFO (Last In, First Out)	Push/Pop	Simple LIFO	No random access	Undo/redo, recursive tasks
HASHTABLE (NON-GENERIC)	Key-value storage	Key must be unique	Fast look up by key	Not type safe, boxing	Legacy code

TYPE	Used for	Features	✓ Pros	✗ Cons	Recommended Case
DICTIONARY <TKEY, TVALUE>	Key-value (generic)	Strong typing	Fast O(1) lookup	No ordering	Most common key-value store
SORTEDLIST (NON-GENERIC)	Key-value + sorting	Auto sorted keys	Sorted lookup	Boxing/unboxing	Legacy
SORTEDLIST <TKEY, TVALUE>	Key-value + sorting	Keeps items sorted by key	Uses less memory	Slower insertion	Small sorted datasets
SORTED DICTIONARY <TKEY, TVALUE>	Sorted dictionary	Tree-based, sorted order	Fast inserts/removals	Slightly higher memory	Large, sorted data lookups
HASHSET	Unique values	No duplicates allowed	Fast O(1) operations	No order guarantee	Store unique items, e.g. tags
OBSERVABLE COLLECTION	Collection with notifications	Raises events when changed	Useful in WPF/Binding	Slight overhead	UI binding (WPF, MVVM)
CONCURRENT COLLECTIONS (CONCURREN TDICTIONARY, BLOCKINGCO LLECTION)	Thread-safe collections	For multi-threaded apps	Thread safety built-in	More overhead	Parallel/async code

4. Which to Use (Most Recommended)

- **List** → Best general-purpose collection.
- **Dictionary<TKey, TValue>** → Best for fast lookups by key.
- **Queue** → Best for FIFO tasks.
- **Stack** → Best for LIFO tasks.
- **LinkedList** → Best when inserting/removing in the middle frequently.
- **HashSet** → Best for unique values, fast search.
- **SortedDictionary<TKey, TValue>** → Best for keeping key-value pairs sorted.
- **ObservableCollection** → Best for UI binding scenarios.

COLLECTION	ACCESS (BY INDEX/KEY)	SEARCH (BY VALUE/CONDITION)	TIME COMPLEXITY (ACCESS)	TIME COMPLEXITY (SEARCH)	NOTES
ARRAY / LIST<T>	list[i] (index)	list. Contains(item), list. Find(...)	O(1) (direct index)	O(n) (linear search)	Best for random access; slow search if value unknown.
LINKEDLIST<T>	(no index access)	Loop through nodes	✗	O(n)	Efficient insert/remove, poor search.
QUEUE<T> / STACK<T>	(no index access)	Contains(item)	✗	O(n)	Optimized for FIFO/LIFO, not searching.
DICTIONARY <TKEY,TVALUE>	dict[key]	dict. ContainsValue(value)	O(1) (hash lookup)	O(n)	Fast key lookup; searching by value is slow.
SORTEDDICTIONARY <TKEY,TVALUE>	dict[key]	Loop through values	O(log n)	O(n)	Keeps keys sorted; slower than Dictionary for access.
SORTEDLIST <TKEY,TVALUE>	list[key]	Loop through values	O(log n)	O(n)	Indexable + keeps order, but slower inserts.
HASHSET<T>	(no index/key)	set. Contains(item)	✗	O(1)	Fast existence check, no duplicates.
OBSERVABLECOLLECTION<T>	collection[i]	Contains(item), IndexOf(item)	O(1) indexAccess	O(n)	Like List<T> but with change notifications (for UI).

- **Fastest Access (O(1)):** Array, List<T>, Dictionary<TKey,TValue>.
- **Fastest Search (O(1)):** Dictionary<TKey,TValue> (by key), HashSet<T> (by value existence).
- **Sorted Search/Access (O(log n)):** SortedDictionary, SortedList.